1️⃣

Week 1 — Digital Images, Linear Algebra & Convolution

Assignment Released	☐
Checkpoint Done	☐
End Date	April 26, 2026
Learning Objectives	Understand image representation, perform matrix operations, apply convolution and basic filters in OpenCV
Phase	⛔ Phase 1 – Foundation
Start Date	April 20, 2026
Status	In Progress
Topics Overview	Digital Images, Image as Matrix, Pixels & Intensity, Linear Algebra in CV, Convolution Basics, Kernels & Filters, Edge Detection
# Week Number	1

⭐ ⭐ **STAR UNION AI ACADEMY — Week 1**
📅 20 Apr – 26 Apr 2026 · Phase 1: Foundation

🎯 Week 1 Objectives

- Understand what a digital image is from a mathematical perspective
- Represent images as 2D and 3D matrices
- Understand pixel values, intensity, and channels

- Grasp the role of linear algebra in Computer Vision
- Learn the concept of convolution and apply it with filters
- Implement basic image operations using OpenCV (Python)

📌 Topic 1: Digital Images & Linear Algebra

📖 Explanation

What is an Image in Computer Vision?

In Computer Vision, an image is not merely a photo — it is a *structured grid of numerical values* that a computer processes mathematically. Every image is made up of tiny units called **pixels** (picture elements), each holding a numerical value that represents light intensity at that point.

Grayscale Images:

A grayscale image is a 2-dimensional matrix of shape `(Height × Width)`. Each cell contains a single integer value from **0** (pure black) to **255** (pure white). For example, a 480×640 grayscale image is a matrix with 307,200 numerical values.

Color (RGB) Images:

A color image has 3 separate channels — **Red, Green, Blue** — stacked together. Its shape becomes `(Height × Width × 3)`. Each channel is its own grayscale matrix. Combining the three channels produces the full-color image.

Pixel Intensity Values:

For an 8-bit image, each pixel stores a value from 0–255:

- 0 = fully dark (black)
- 255 = fully bright (white or full color)
- Values in between represent shades or intermediate colors

Role of Linear Algebra in Computer Vision:

Linear algebra is the mathematical foundation of nearly every CV operation:

- **Matrices** represent images
- **Matrix multiplication** transforms images (rotation, scaling, perspective)
- **Vectors** represent pixel neighborhoods or feature embeddings

- **Dot products** measure similarity between image patches
- **Eigenvalues/SVD** are used in dimensionality reduction (PCA) and feature analysis
- **Convolution** is a form of matrix cross-correlation

Image Coordinate System:

Notation matters. In NumPy/OpenCV, an image array has shape `(rows, cols, channels)`, where rows = Height and cols = Width. The origin (0,0) is at the **top-left** corner, with Y increasing **downward**.

Key NumPy Operations on Images:

- `img.shape` → (H, W, C)
- `img[y, x]` → pixel at row y, column x
- `img[y, x, 0]` → Blue channel value (OpenCV uses BGR order)
- `img.dtype` → usually `uint8` (8-bit unsigned integer)

🔗 References

- [Add link: OpenCV Official Documentation — https://docs.opencv.org]
- [Add link: NumPy for Image Processing — https://numpy.org/doc]
- [Add link: Linear Algebra for Machine Learning (3Blue1Brown) — YouTube]
- [Add link: CS231n Stanford — Image Classification Notes]
- [Add link: Python OpenCV Tutorial — Real Python]

🧠 Summary

- A digital image = a 2D (grayscale) or 3D (color) **matrix of pixel values**
- Each pixel = a numerical intensity value between **0 and 255** for 8-bit images
- RGB images have 3 channels; OpenCV uses **BGR** channel order by default
- Linear algebra operations (matrix multiplication, dot product, eigenvalues) are **core tools** in every CV algorithm
- Image shape in NumPy: `(Height, Width, Channels)`
- The coordinate origin is at the **top-left**; Y increases downward

❓ Theory Questions

1. What is a pixel, and what values can it hold in an 8-bit image?
2. How is a grayscale image different from a color (RGB) image in terms of matrix representation?
3. What does the shape `(480, 640, 3)` mean for an image array?
4. Why does OpenCV use BGR instead of RGB channel ordering?
5. How is matrix multiplication used in image transformations such as rotation?
6. What is the role of eigenvalues in computer vision applications?
7. Explain the difference between `img[y, x]` and `img[x, y]` and why the ordering matters.
8. What is the range of pixel values for a 16-bit image?

💻 Coding Task 1

```python
# Task: Load, Inspect, and Analyze an Image using OpenCV

import cv2
import numpy as np

# Step 1: Load image (provide your own image path)
img = cv2.imread('sample.jpg')

# Step 2: Print shape, dtype, and pixel stats
print("Shape:", img.shape)          # (H, W, C)
print("Data type:", img.dtype)      # uint8
print("Min pixel:", img.min())
print("Max pixel:", img.max())
print("Mean pixel:", img.mean())

# Step 3: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print("Grayscale shape:", gray.shape)  # (H, W)
```

```
# Step 4: Print a 5x5 patch of grayscale values from top-le
ft
print("Top-left 5x5 pixel values:\n", gray[0:5, 0:5])

# Step 5: Display both images
cv2.imshow('Original', img)
cv2.imshow('Grayscale', gray)
cv2.waitKey(0)
cv2.destroyAllWindows()

# Step 6: Normalize to 0-1 range (float32)
normalized = gray.astype(np.float32) / 255.0
print("Normalized range:", normalized.min(), "-", normalize
d.max())
```

Expected outcomes:

- Image loads and displays correctly in both color and grayscale
- Shape, dtype, min/max/mean values are printed
- A 5×5 pixel value patch is visible in the console
- Normalized image values are between 0.0 and 1.0

📌 Topic 2: Convolution Basics

📖 Explanation

What is Convolution?

Convolution is the **fundamental mathematical operation** behind almost all image processing and deep learning in CV. It is the process of sliding a small matrix (called a **kernel** or **filter**) across an image and computing weighted sums at every position.

The Kernel (Filter):

A kernel is a small 2D matrix — typically 3×3, 5×5, or 7×7 — containing numerical weights. These weights define what feature the convolution will detect or enhance. For example:

- A **blur kernel** averages neighboring pixels

- An **edge detection kernel** highlights rapid intensity changes
- A **sharpening kernel** enhances fine details

How Convolution Works (Step by Step):

1. Place the kernel over a region of the image (starting at the top-left)
2. Multiply each kernel weight by the corresponding image pixel value
3. Sum all the products → this is the **output pixel value**
4. Slide the kernel one step (stride=1 by default) to the right
5. Repeat for every position across the entire image
6. The result is a new image called the **feature map** or **activation map**

Mathematical Formula:

`(I * K)[i,j] = Σₘ Σₙ I[i+m, j+n] × K[m,n]`

where I = image, K = kernel, (i,j) = output position.

Common Kernels:

Kernel Name	Purpose	Example 3×3
Identity	No change	[[0,0,0],[0,1,0],[0,0,0]]
Box Blur	Smooth/average	all values = 1/9
Gaussian Blur	Smooth (weighted)	bell-curve weights
Sobel X	Horizontal edges	[[-1,0,1],[-2,0,2],[-1,0,1]]
Sobel Y	Vertical edges	[[-1,-2,-1],[0,0,0],[1,2,1]]
Laplacian	All-direction edges	[[0,1,0],[1,-4,1],[0,1,0]]
Sharpen	Enhance detail	[[0,-1,0],[-1,5,-1],[0,-1,0]]

Padding:

When you convolve without padding, the output shrinks. `SAME` padding (adding zeros around the border) keeps the output the same size as the input. `VALID` padding means no padding, output is smaller.

Stride:

Stride controls how many pixels the kernel moves at each step. Stride=1 → dense sampling. Stride=2 → output is half the size.

Why Convolution is Core to CV:

In deep learning, CNNs learn their kernels automatically during training. Each convolutional layer learns to detect increasingly complex features — edges → textures → shapes → objects. This hierarchical feature learning is what makes CNNs powerful.

🔗 References

- [Add link: OpenCV Filtering Docs — https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html]
- [Add link: CS231n Convolutional Networks — http://cs231n.github.io/convolutional-networks]
- [Add link: 3Blue1Brown — But what is a convolution? — YouTube]
- [Add link: NumPy convolve docs — https://numpy.org/doc/stable/reference/generated/numpy.convolve.html]

🧠 Summary

- Convolution = sliding a **kernel** over an image and computing **weighted sums**
- The kernel defines what **feature** is detected (blur, edges, sharpen, etc.)
- Output = **feature map** of same or smaller size depending on padding
- Padding preserves size; stride controls output resolution
- This operation is the **building block of all CNNs**
- Common kernels: Box Blur, Gaussian Blur, Sobel, Laplacian, Sharpen

---

❓ Theory Questions

1. Define convolution in your own words. How is it different from simple multiplication?
2. What is a kernel/filter? Give two examples and explain what each one detects.
3. What is the difference between SAME and VALID padding?
4. If you convolve a 100×100 image with a 3×3 kernel (no padding, stride=1), what is the output size?
5. Why does a Sobel X filter detect horizontal edges?

6. What happens when all kernel values are identical (e.g., all = 1/9)?
7. How does stride affect the size of the output feature map?
8. Why does CNN learn kernels automatically during training?

💻 Coding Task 2

```
# Task: Apply and Compare Multiple Convolution Filters

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load and convert to grayscale
img = cv2.imread('sample.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# --- Define kernels ---

# 1. Box blur (averaging)
kernel_blur = np.ones((5, 5), np.float32) / 25

# 2. Gaussian blur (built-in)
# (done via cv2.GaussianBlur)

# 3. Sobel edge detection (X direction)
kernel_sobel_x = np.array([[-1, 0, 1],
                           [-2, 0, 2],
                           [-1, 0, 1]], dtype=np.float32)

# 4. Laplacian edge detection
kernel_laplacian = np.array([[0,  1, 0],
                             [1, -4, 1],
                             [0,  1, 0]], dtype=np.float3
2)

# 5. Sharpening
kernel_sharpen = np.array([[ 0, -1,  0],
```

```
                              [-1,  5, -1],
                              [ 0, -1,  0]], dtype=np.float3
2)

# --- Apply filters ---
blurred    = cv2.filter2D(gray, -1, kernel_blur)
gaussian   = cv2.GaussianBlur(gray, (5, 5), 0)
edge_x     = cv2.filter2D(gray, -1, kernel_sobel_x)
laplacian  = cv2.filter2D(gray, -1, kernel_laplacian)
sharpened  = cv2.filter2D(gray, -1, kernel_sharpen)

# --- Display all results ---
titles  = ['Original', 'Box Blur', 'Gaussian Blur', 'Sobel
X', 'Laplacian', 'Sharpened']
images  = [gray, blurred, gaussian, np.abs(edge_x).astype(n
p.uint8),
           np.abs(laplacian).astype(np.uint8), sharpened]

plt.figure(figsize=(15, 8))
for i, (t, im) in enumerate(zip(titles, images)):
    plt.subplot(2, 3, i+1)
    plt.imshow(im, cmap='gray')
    plt.title(t)
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Expected outcomes:

- 6 images displayed side by side
- Blurred images show smoothing effect
- Sobel X shows vertical lines/edges highlighted
- Laplacian shows all edges
- Sharpened image shows enhanced fine details

🧪 Weekly Checkpoint — CP1: Image Fundamentals

🧪 **Pass Mark: 70% — Both theory and coding sections are graded**

❓ Theory Section (50 pts)

1. (10 pts) What is a pixel? Explain intensity values for 8-bit images.
2. (10 pts) What is the shape of a color image with H=480, W=640? Explain each dimension.
3. (15 pts) Explain convolution step by step. Include kernel, sliding window, and output.
4. (15 pts) Compare Box Blur vs Gaussian Blur vs Sobel — purpose and kernel differences.

💻 Coding Section (50 pts)

Task: Build a Simple Image Filter Tool

Build a Python script that:

1. Loads any image from disk
2. Converts it to grayscale
3. Applies at least 3 different filters (blur, edge detection, sharpen)
4. Saves each output image with a descriptive filename
5. Prints image shape and pixel statistics for each result

Bonus (10 pts): Add a Canny edge detector (`cv2.Canny`) and compare its output with the Laplacian filter.

📝 Week 1 Assignment

Title: Image Analysis & Filter Exploration

Deadline: 26 Apr 2026

Description:

Choose 3 different types of real-world images (e.g., portrait, landscape, document). For each image:

1. Load and display the image with OpenCV
2. Print its shape, dtype, and pixel statistics

3. Apply 5 different filters (blur, Gaussian, Sobel X, Sobel Y, Laplacian)
4. Display all results in a single matplotlib figure
5. Write a short paragraph (100 words) explaining which filter was most effective for each image and why

Required Concepts: Image matrix representation, pixel intensity, convolution, kernels

Submission Format:

- Python `.py` file (well commented)
- Output image screenshots
- Short written analysis (PDF or `.md` file)

Grading Rubric:

- Code runs without errors: 20 pts
- Correct filter application: 30 pts
- Visualization quality: 20 pts
- Written analysis: 30 pts
- **Total: 100 pts**
